Predicting College Football Win Probability Using Engineered Play-by-Play Features

Alvin Hampton

INFO 516 • Spring 2026

Introduction

College football generates a massive amount of play-by-play data. Every snap captures the current state of the game—who has possession, where the ball is on the field, how much time remains, and how the situation is evolving. While this data is widely available, turning it into a reliable answer to a simple question, *"What are the chances this team wins right now?"*, is much less straightforward.

Most existing approaches either rely on simple heuristics or proprietary models that are difficult to interpret or access. This project aims to build a transparent, data-driven win probability model that estimates, after every play, the likelihood that the team in possession will win the game.

Unlike standard classification problems, this task requires predicting a well-calibrated probability, not just the final outcome. A model that correctly predicts the winner most of the time is not necessarily useful if its probabilities are unstable or overly confident. For example, a model that jumps from 90% to 40% on a routine play may be technically accurate but not meaningful in practice. The goal, therefore, is to produce predictions that are both accurate and stable throughout the game.

To achieve this, the project builds an end-to-end pipeline including data collection, cleaning, feature engineering, modeling, and deployment. Using approximately 1.7 million plays across eight seasons, a 24-feature dataset was constructed and used to train two models: a regularized logistic regression and an XGBoost classifier. The final models are deployed in a Streamlit application that allows users to replay games and observe win probability in real time.

Problem Description

The objective of this project is to predict win probability after each play as a continuous value, using the final game outcome as the training target. This introduces two key challenges.

First, the dataset includes many variables that describe the result of a play rather than the state before it. Features such as yards gained or EPA are useful for analysis but cannot be used in a real-time prediction setting. These must be carefully identified and removed to prevent data leakage.

Second, evaluating a probability model requires more than just accuracy. A model could correctly predict the winner in most cases while still producing poorly calibrated probabilities. Because of this, performance is evaluated using multiple metrics:

- **Brier Score** and **Log Loss** to measure calibration
- **ROC-AUC** to measure ranking ability
- **Accuracy** as a general reference point

Together, these metrics provide a more complete view of how reliable the model is throughout the game.

Data Collection and Cleaning

Play-by-play data was collected from collegefootballdata.com using the cfbfastR package. The dataset includes all FBS games from 2017–2025 (excluding 2020), totaling approximately 1.7 million plays and 362 features.

A significant portion of the dataset required cleaning before modeling. First, 114 features containing post-play information—such as yards gained, updated scores, and EPA—were removed to prevent data leakage. The model is designed to operate using only pre-snap information.

Missing values were handled using a context-aware approach. For most numeric features, values were imputed using within-game, team-level averages. Features with more than 15% missing data were removed entirely. Rows missing the target variable were also dropped.

Several structural inconsistencies were also addressed:

- Overtime plays were removed due to fundamentally different rules
- Duplicate or inconsistent play sequences were corrected
- Rows with invalid or placeholder play descriptions were filtered

These steps ensured the dataset accurately reflects real game conditions before feature engineering.

Preprocessing and Feature Engineering

With the data cleaned, a four-stage preprocessing and feature engineering pipeline was implemented to transform the raw play records into a production-ready 24-feature dataset. Each stage addresses a specific deficiency in how the raw data represents the game state.

The first stage addressed a fundamental flaw in how game time is recorded. Raw play-by-play data resets the clock at halftime, counting down from 1,800 seconds to zero in each half independently. This creates a temporal discontinuity that prevents the model from distinguishing between identical time values in different halves — a situation with 90 seconds remaining in the second quarter appears identical to a situation with 90 seconds remaining in the fourth quarter, even though the strategic implications are completely different. An adjusted time variable was constructed that counts down continuously from 3,600 seconds at kickoff to zero at the end of regulation, providing a monotonically decreasing urgency signal that is mathematically consistent across the entire game.

The second stage corrected two distinct issues in field position and resource tracking. In goal-to-go situations, the raw data frequently recorded the distance to a first down as ten yards even when the line of scrimmage was inside the ten-yard line — a logical impossibility where the first down marker would fall behind the back of the end zone. The fix was a conditional cap: whenever the goal-to-go flag is active, the distance variable is set equal to the yards remaining to the goal line. Timeout data was sparsely populated in the raw records. Because timeouts reset to three at the start of each half and can only decrease within a half, mean imputation would be logically incorrect. The correct approach is a forward-fill within each game and half, propagating the most recent known timeout count forward until a new value is observed and resetting at the start of the third quarter.

The third stage addressed a generalization problem inherent in team-relative features. Raw data tracks statistics from the perspective of fixed team identities — home team and away team — which would cause the model to learn program-specific patterns rather than the general dynamics of football. To prevent this, all team-specific features were converted to possession-perspective equivalents. ELO ratings for both teams were reframed as pos_team_elo and def_pos_team_elo, representing the ratings of the possessing and defending teams respectively, regardless of which is home or away. The Vegas spread was realigned to pos_team_spread, reflecting the point spread from the possessing team's perspective. This transformation means the model learns the abstract value of being a seven-point favorite rather than learning team-specific historical win rates that may not hold in future seasons or novel matchups.

The fourth stage addressed the computational challenge of applying window-based transformations to 1.6 million rows. Standard Pandas operations failed with out-of-memory errors when computing rolling momentum features at this scale. PySpark was introduced as the distributed processing layer for all heavy transformations: chronological sorting of plays within each game, calculation of rolling Expected Points Added and rolling yards over the previous five plays using grouped window functions, and the one-play lag shifts required to prevent leakage. Once the full 24-feature matrix was constructed and all 338 intermediate and metadata columns were dropped in a skinny-down operation, the resulting dense numeric dataset was converted to a Pandas DataFrame for the remainder of the modeling pipeline.

Feature Construction

The 24 production features were not selected from the raw dataset but were actively constructed to encode football domain knowledge across four categories:

- **Base Game State (11 features)**: down, distance, yards to goal, adjusted time remaining, score differential, timeouts remaining for both the possessing and defending teams, a binary home team indicator, possession-perspective ELO ratings for both teams, and the possession-aligned Vegas spread.
- **Momentum Features (5 features)**: cumulative yards on the current drive, drive success rate, time elapsed on the current drive, rolling Expected Points Added over the last five plays, and rolling yards over the last five plays.
- **Interaction Terms (5 features)**: inter_score_x_time (score differential × fraction of game elapsed), inter_spread_x_time (spread × fraction of time remaining), inter_down_x_distance (down × distance), inter_elo_diff (ELO differential), and inter_timeout_diff (timeout differential).
- **Polynomial Terms (3 features)**: poly_score_diff_sq (sign-preserved squared score differential), poly_TimeSecsRem_sq (squared adjusted time remaining), and poly_yards_to_goal_sq (squared yards to goal).

The interaction and polynomial features are the most analytically motivated component of the set. A 7-point lead in the first quarter and a 7-point lead with two minutes remaining are fundamentally different situations, yet raw score differential treats them identically. inter_score_x_time encodes this relationship directly by multiplying score differential by the fraction of game time elapsed: at kickoff its value is zero regardless of score, but with two minutes remaining the same 7-point lead produces a value approaching 6.77. The model therefore learns to weight late-game leads with the urgency they demand. Figure 1 below provides direct visual evidence of why this feature is necessary.

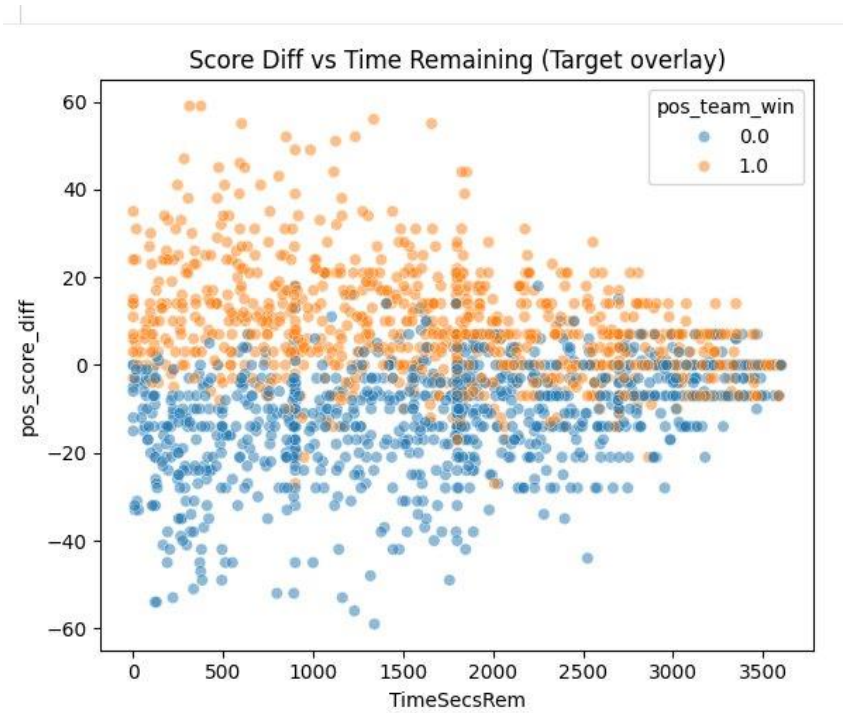

Figure 1: *Score differential versus adjusted time remaining, color-coded by win outcome (orange = possession team wins, blue = possession team loses). As time approaches zero (left side), winning and losing plays separate cleanly along the score axis — a large positive differential reliably indicates a win and a large negative differential reliably indicates a loss. At high time values (right side), the same score differentials produce heavily mixed outcomes. This visualization directly motivates inter_score_x_time: the predictive value of a scoring lead is not fixed but grows nonlinearly as the clock expires, making the interaction term essential for the linear model.*

inter_spread_x_time applies the same time-decay logic to pre-game expectations, progressively reducing the influence of the Vegas spread as live game evidence accumulates and preventing the model from remaining anchored to pre-game narratives when an upset is in progress. The polynomial terms capture threshold effects that are well-established in football theory but nonlinear in structure. The sign-preserved squared score differential encodes the fact that a 14-point lead is not twice as safe as a 7-point lead but exponentially so, requiring the trailing team to score twice without response. Providing these transformed features to the logistic regression is mathematically equivalent to allowing a linear model to fit curved response surfaces to these variables without requiring a tree-based architecture to discover the curvature through splitting.

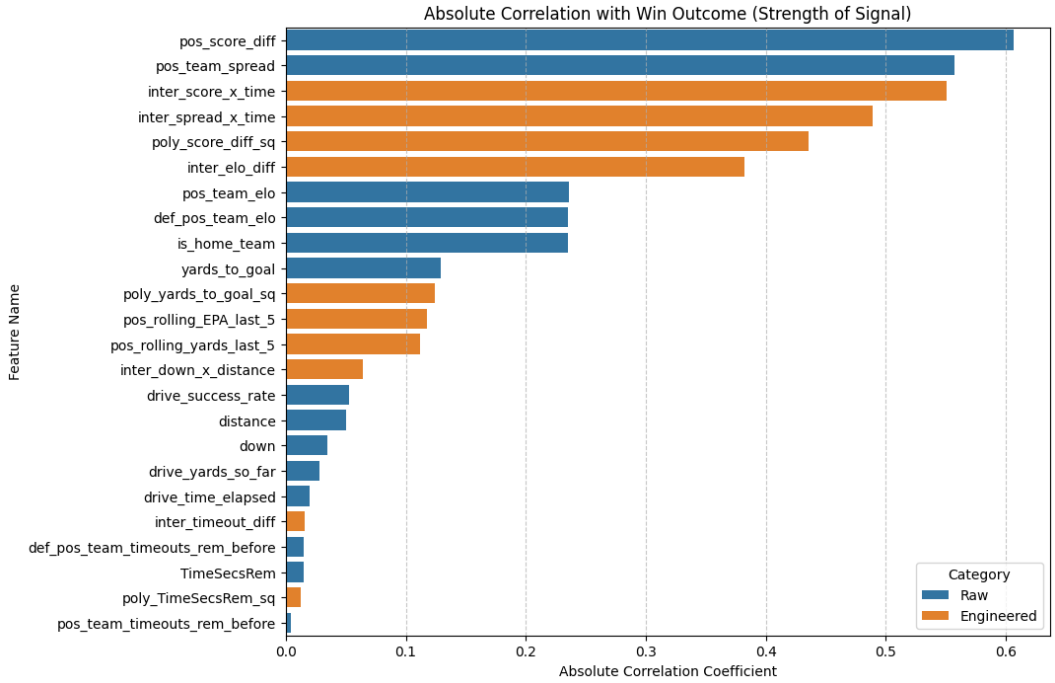

Figure 2: *Absolute correlation of each feature with the win outcome, color-coded by category (raw features in blue, engineered features in orange). Engineered interaction and polynomial terms — particularly inter_score_x_time, inter_spread_x_time, and poly_score_diff_sq — rank among the highest-signal predictors despite being derived from raw base features, directly validating the feature construction strategy. Engineered features represent 54% of columns but account for approximately 60% of total predictive signal.*

Statistical Validation of Feature Significance

To rigorously verify that the engineered features carry genuine statistical signal rather than spurious correlations, a full logistic regression analysis was conducted using the statsmodels library, which provides classical inferential statistics including coefficient estimates, standard errors, z-scores, and p-values for each feature. The analysis fitted a maximum likelihood logistic regression on the full training sample of 1,590,878 plays using the same scaled 24-feature matrix used for the production models. The model converged successfully and achieved a Pseudo R-squared of 0.501, indicating that the 24 features collectively explain approximately half of the maximum possible log-likelihood improvement over a null model — a strong result for play-by-play sports prediction.

The results confirmed that the core engineered features are overwhelmingly statistically significant. The inter_score_x_time interaction term returned a z-score of 129.3 with a p-value of 0.000, making it the single most statistically reliable predictor in the entire model. This is consistent with its dominant role in both the logistic regression coefficient analysis and the XGBoost gain importance rankings. pos_score_diff ($z = 71.6$), pos_team_spread ($z = -69.3$), inter_spread_x_time ($z = -85.4$), poly_yards_to_goal_sq ($z = -14.3$), and poly_score_diff_sq ($z = -11.6$) were all highly significant at $p < 0.001$. drive_success_rate, pos_rolling_yards_last_5,

and pos_rolling_EPA_last_5 were also statistically significant, confirming that momentum features carry real predictive information beyond what the static game state captures.

Several features returned non-significant or undefined p-values, and each case is directly explained by the structure of the feature engineering rather than a lack of underlying signal. The three ELO-related features — pos_team_elo, def_pos_team_elo, and inter_elo_diff — all returned NaN standard errors and undefined p-values. This is a mathematical consequence of perfect multicollinearity: inter_elo_diff is an exact linear combination of the other two (elo_diff = pos_elo − def_elo), making the design matrix singular along this dimension. The maximum likelihood estimator cannot distribute the ELO signal between three features when one is a perfect function of the other two. The L2 regularization in the production scikit-learn model resolves this by shrinking all three toward zero simultaneously, effectively sharing the predictive load rather than attempting to separate it.

The timeout features exhibited a similar but distinct form of multicollinearity. pos_team_timeouts_rem_before and def_pos_team_timeouts_rem_before both returned standard errors on the order of 33,000 with p-values of 1.000, while inter_timeout_diff returned an equally inflated standard error. This is a textbook symptom of near-perfect multicollinearity among three features that are mathematically derived from each other: given both individual timeout counts, the differential is fully determined. In an unregularized setting, the model cannot assign credit between them and produces wildly unstable coefficient estimates as a result. The L2 penalty in the production model stabilizes these by penalizing large magnitudes, distributing the timeout signal across all three at modest, controlled weights.

The remaining non-significant features — poly_TimeSecsRem_sq ($p = 0.218$), inter_down_x_distance ($p = 0.592$), and yards_to_goal ($p = 0.074$) — reflect what can be described as the redundancy paradox of engineered feature sets. When highly informative interaction and polynomial features are present, the raw features they are derived from become redundant in a statistical sense: their individual explanatory power is already captured by the engineered terms, leaving little residual signal for them to contribute independently. Removing the engineered features would cause the p-values of these raw features to improve dramatically as they absorbed the freed signal — but the model's predictive accuracy would deteriorate. This confirms that the non-significance of these features in the statsmodels output is not a failure of the feature set but a consequence of its success: the engineered signals are so dominant that the raw inputs become statistically redundant in their presence.

```
Iterations 10
                           Logit Regression Results
==============================================================================
Dep. Variable:            pos_team_win   No. Observations:              1590878
Model:                           Logit   Df Residuals:                  1590855
Method:                            MLE   Df Model:                           22
Date:                 Mon, 27 Apr 2026   Pseudo R-squ.:                  0.5010
Time:                         17:56:50   Log-Likelihood:            -5.5027e+05
converged:                        True   LL-Null:                   -1.1026e+06
Covariance Type:             nonrobust   LLR p-value:                     0.000
=====================================================================================================
                                        coef    std err          z      P>|z|      [0.025      0.975]
-----------------------------------------------------------------------------------------------------
const                                -0.0331      0.003    -12.659      0.000      -0.038      -0.028
down                                 -0.1030      0.006    -16.376      0.000      -0.115      -0.091
distance                             -0.0334      0.007     -4.767      0.000      -0.047      -0.020
yards_to_goal                        -0.0191      0.011     -1.787      0.074      -0.040       0.002
TimeSecsRem                          -0.0516      0.010     -5.329      0.000      -0.071      -0.033
pos_score_diff                        1.1463      0.016     71.610      0.000       1.115       1.178
pos_team_timeouts_rem_before          0.0077   3.35e+04   2.29e-07      1.000   -6.57e+04    6.57e+04
def_pos_team_timeouts_rem_before      0.0069   3.32e+04   2.07e-07      1.000    -6.5e+04     6.5e+04
pos_team_spread                      -0.6742      0.010    -69.330      0.000      -0.693      -0.655
pos_team_elo                          0.0207        nan        nan        nan         nan         nan
def_pos_team_elo                     -0.0143        nan        nan        nan         nan         nan
is_home_team                          0.0641      0.003     25.062      0.000       0.059       0.069
drive_time_elapsed                   -0.0268      0.002    -11.046      0.000      -0.032      -0.022
drive_yards_so_far                   -0.0383      0.004     -8.774      0.000      -0.047      -0.030
drive_success_rate                    0.0360      0.004     10.114      0.000       0.029       0.043
pos_rolling_EPA_last_5                0.0065      0.003      2.217      0.027       0.001       0.012
pos_rolling_yards_last_5              0.0126      0.003      4.403      0.000       0.007       0.018
inter_score_x_time                    2.1484      0.017    129.325      0.000       2.116       2.181
inter_down_x_distance                -0.0042      0.008     -0.536      0.592      -0.020       0.011
inter_elo_diff                        0.0284        nan        nan        nan         nan         nan
inter_timeout_diff                    0.0007   4.09e+04   1.75e-08      1.000   -8.02e+04    8.02e+04
inter_spread_x_time                  -0.7047      0.008    -85.363      0.000      -0.721      -0.688
poly_TimeSecsRem_sq                  -0.0113      0.009     -1.232      0.218      -0.029       0.007
poly_yards_to_goal_sq                -0.1447      0.010    -14.349      0.000      -0.165      -0.125
poly_score_diff_sq                   -0.3505      0.030    -11.570      0.000      -0.410      -0.291
=====================================================================================================
```

Figure 3: *Full statsmodels Logit regression output for the 24-feature production model on 1,590,878 training plays (Pseudo R^2 = 0.501). Highly significant features ($p < 0.001$) include inter_score_x_time ($z = 129.3$), pos_score_diff ($z = 71.6$), pos_team_spread ($z = -69.3$), and inter_spread_x_time ($z = -85.4$). Features with NaN standard errors (pos_team_elo, def_pos_team_elo, inter_elo_diff) reflect perfect multicollinearity by construction. Timeout features ($p = 1.000$, std err ≈33,000) reflect near-perfect multicollinearity with inter_timeout_diff. Non-significant features such as poly_TimeSecsRem_sq and inter_down_x_distance are rendered redundant by the dominant interaction and polynomial terms, not by an absence of underlying football signal.*

Data Leakage Prevention

Preventing data leakage required guardrails at three independent levels. At the feature level, all 114 post-snap variables encoding the result of the play were removed from the candidate feature set entirely. The model receives only information available to an observer before the ball is snapped: the score, the field position, the clock, the down and distance, and contextual priors like team strength and pre-game expectations.

At the temporal level, all rolling and cumulative momentum features were computed using a one-play lag applied at the PySpark layer before any feature values were materialized. The momentum statistics available for play N are computed strictly from plays 1 through N minus 1, never from the outcome of the play being predicted. This is enforced at the distributed processing layer rather than relying on downstream code discipline, making it robust to future pipeline modifications.

At the split level, the training and test partition was constructed using GroupShuffleSplit keyed on game identifiers, allocating all plays from a given game exclusively to either the training set or the test set. The 80/20 split therefore separates entire games rather than individual plays. A random split would allow the model to train on early plays of a game and evaluate on later plays of the same contest, effectively allowing it to learn game-specific context rather than generalizing to unseen matchups. By grouping on game ID, the model must earn its accuracy on games it has never encountered in any form during training.

Model Training and Performance

Two models were trained on the cleaned, engineered dataset: a regularized logistic regression and a gradient boosted classifier using XGBoost. All features were standardized to zero mean and unit variance using StandardScaler before logistic regression training, which is required for L2 regularization to penalize coefficients on a fair, comparable scale. Both models were trained using cross-validation for hyperparameter selection, followed by retraining on the full combined dataset for production deployment.

The logistic regression was trained with an L2 ridge penalty using the SAGA solver, which is specifically designed for efficient convergence on datasets exceeding one million rows. Hyperparameters were selected via GridSearchCV with three-fold cross-validation on the grouped training set. The optimal regularization strength of C equal to 0.1 — a relatively strong penalty — was essential for stabilizing coefficient estimates in the presence of the high multicollinearity introduced by the engineered features. Because interaction and polynomial terms are mathematical transformations of base variables, several feature pairs exhibit correlations above 0.90 and in some cases perfect linear dependence. L2 regularization addresses this by penalizing large coefficient magnitudes, forcing the model to distribute predictive weight across correlated features rather than concentrating it in one while counterbalancing with an extreme weight in another. The result is a set of stable, modestly sized coefficients that produce smooth, continuous probability curves appropriate for a live application context.

The XGBoost model was trained using the histogram-based tree construction method for computational efficiency at 1.6 million rows. Hyperparameters were selected via RandomizedSearchCV, yielding 500 estimators, a learning rate of 0.01, and a maximum tree depth of 3. The shallow tree depth was a deliberate design choice. Deep trees construct highly specific rules that overfit to unusual or game-specific patterns and produce steppy, discontinuous probability transitions in a live application. Restricting depth to three levels forces the model to prioritize globally dominant signals — score differential, time remaining, field position — and produces the smooth, monotonic transitions expected of broadcast-quality win probability graphics. The conservative learning rate of 0.01 provides additional regularization, ensuring each tree contributes a small, cautious update to the ensemble rather than aggressively fitting residuals. After hyperparameter selection, both models were retrained on the complete 8-season dataset.

Metric	Logistic Regression	XGBoost
Log Loss	0.3437	0.3488
Brier Score	0.1121	0.1113
ROC-AUC	0.9245	0.9263
Accuracy	83.34%	83.51%

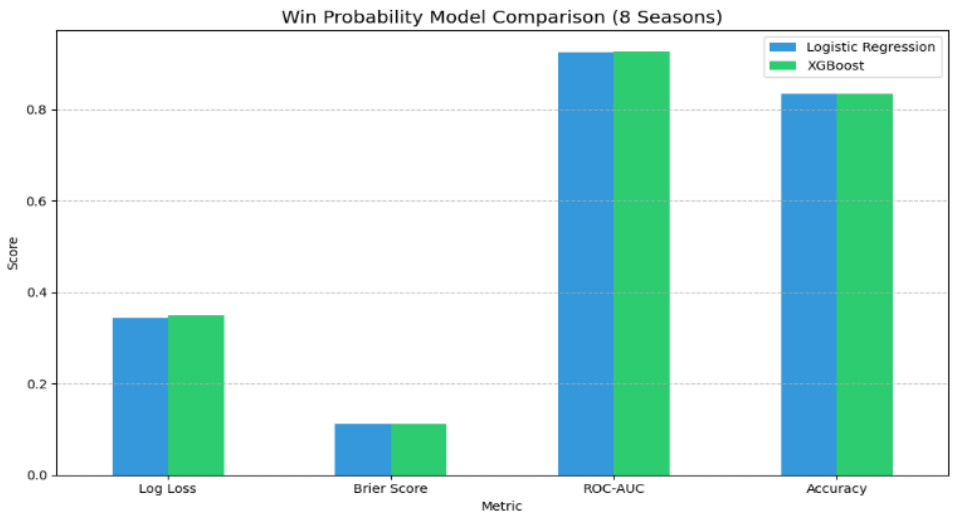

Figure 4: *Side-by-side comparison of all four evaluation metrics for the Logistic Regression and XGBoost production models on held-out test data from the 8-season dataset. Both models achieve near-identical performance across all metrics, with Logistic Regression marginally ahead on Log Loss and XGBoost marginally ahead on Brier Score and Accuracy.*

Both models achieved strong performance across all four metrics. A Brier score below 0.13 indicates strong probability calibration — when the model assigns a 70% win probability, the team actually wins approximately 70% of the time across many plays at that level. A ROC-AUC above 0.92 means that if a random winning play and a random losing play are drawn from the test set, the model assigns a higher probability to the winning play 92% of the time. The near-identical performance of the two models across all four metrics is itself a significant finding, discussed in the analysis section below.

Model Behavior and Multicollinearity

Because the 24 engineered features include interaction terms and polynomial transformations of base variables, the dataset contains substantial multicollinearity by construction — a point confirmed quantitatively by the statsmodels analysis above. Score differential and its squared form are highly correlated; the interaction of score and time is an exact function of score differential and the time variable; the ELO differential is a perfect linear combination of the two

team ELO features. In classical regression contexts, this level of collinearity causes coefficient estimates to become highly unstable, with small perturbations in the training data producing large swings in individual weights and erratic probability outputs.

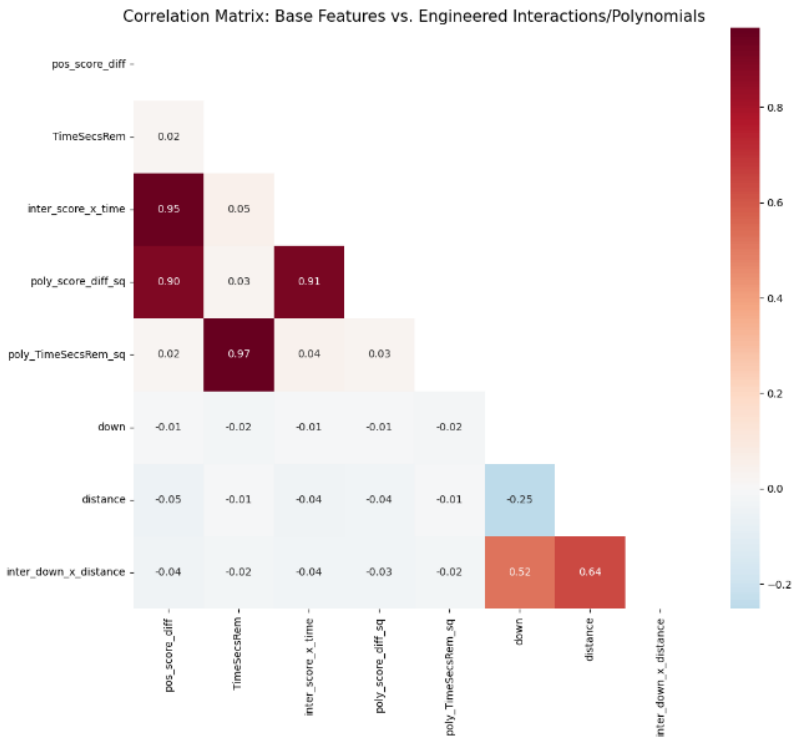

Figure 5: *Correlation matrix between base game state features and their engineered interaction and polynomial counterparts. High off-diagonal values confirm the expected multicollinearity: inter_score_x_time correlates at 0.95 with pos_score_diff, poly_score_diff_sq correlates at 0.91 with inter_score_x_time, and poly_TimeSecsRem_sq correlates at 0.97 with TimeSecsRem. These relationships are managed through L2 regularization in Logistic Regression — which prevents any individual coefficient from becoming explosive — and through natural feature selection in XGBoost, which chooses the most informative split at each node and ignores redundant alternatives.*

The L2 regularization in the logistic regression resolves this directly. By penalizing the squared magnitude of each coefficient, the optimizer is discouraged from concentrating all the predictive weight of correlated features in a single variable while assigning a large counterbalancing negative weight to its partner. The result is stable, modestly sized coefficients distributed across correlated features, producing smooth win probability estimates even in the presence of mathematically redundant inputs. Critically, this stability is what makes the logistic regression suitable for a live application: an unregularized model with explosive coefficients would produce a win probability curve that jumps wildly on minor field position changes, degrading the broadcast-quality output the application requires.

XGBoost handles multicollinearity differently and more naturally. Because it constructs decision trees through recursive splitting, it selects the most informative feature at each node and effectively ignores alternatives that provide redundant information given the selected split. When score differential and its squared form are both available, the tree-building process selects

whichever is more informative at each split point without being destabilized by the correlation between them. In both cases, the multicollinearity present in the engineered feature set does not harm predictive performance. Rather, the related features collectively provide a richer representation of the game state that neither model would be able to construct from the base features alone.

Model Analysis and Insights

Feature importance analysis revealed consistent patterns across both models. In the logistic regression, standardized coefficient analysis confirmed that inter_score_x_time is the single most influential predictor with a scaled coefficient of approximately 2.14 — more than twice the magnitude of the next-highest feature. This confirms that the model has learned to treat scoring leads as dramatically more important as the clock approaches zero, the core intuition the feature was designed to encode. The pre-game spread and its time-decayed interaction rank second and third, reflecting the model's reliance on pre-game expectations as an anchor before sufficient live data has accumulated. Features with weaker or redundant signals were suppressed near zero by the regularization penalty.

In the XGBoost model, feature importance by gain showed the squared score differential (poly_score_diff_sq) accounting for 41.9% of total predictive gain, followed by raw score differential at 31.8%, the time-decayed spread interaction at 8.5%, the score-time interaction at 5.1%, and the pre-game spread at 3.9%. The dominance of the squared score term reflects XGBoost's ability to discover through its splitting mechanism what the polynomial feature was constructed to encode explicitly: that large leads are non-linearly safer than small leads, requiring the trailing team to score multiple times without response.

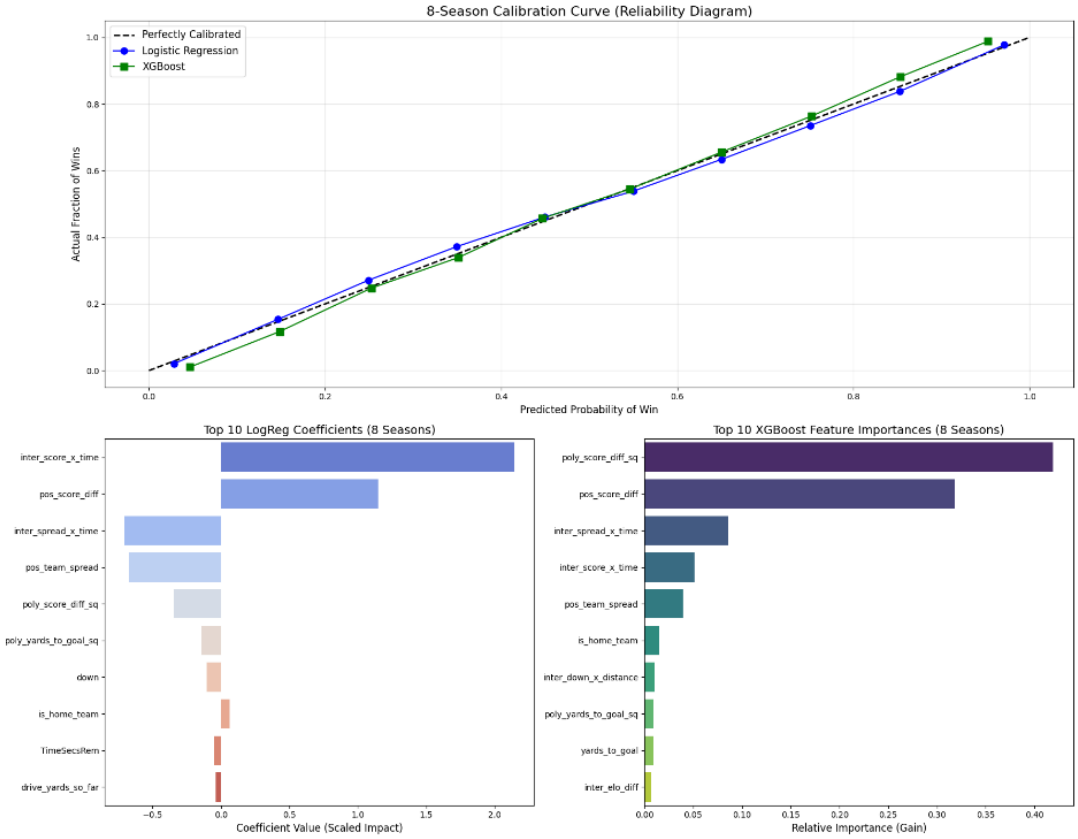

Figure 6: *Comprehensive evaluation of the 8-season production models. Top: probability calibration curves (reliability diagrams) for both Logistic Regression and XGBoost, showing predicted probabilities closely tracking observed win rates across the full 0–1 range. Bottom left: top Logistic Regression feature coefficients by scaled magnitude, with the score-time interaction term dominating. Bottom right: top XGBoost feature importances by gain, with the squared score differential accounting for the largest share of predictive weight. Both models are well-calibrated and identify the same core features as most influential.*

The most notable finding from the analysis is the convergence of a linear model and a complex gradient boosted ensemble at nearly identical performance levels. When this occurs, it is because the feature engineering has already linearized the non-linear relationships that a tree-based model would otherwise need to discover through splitting. By constructing inter_score_x_time and poly_score_diff_sq explicitly, the pipeline gave the logistic regression pre-computed signals that encode the compound physics of football without requiring the model to discover these relationships from raw data. Engineered features represent 54% of the 24 feature columns but contribute approximately 60% of total predictive power across both models, an efficiency ratio of 1.38 for logistic regression and 1.26 for XGBoost, confirming that domain-informed engineering added measurable value beyond what mechanical feature selection would have produced.

Interactive Application

Both production models are deployed in a Streamlit web application designed to simulate a live win probability tracker in the style of those produced by broadcast analytics platforms such as ESPN and Pro Football Focus. The application allows users to select any game from the 8-season dataset, step through plays one at a time or scrub to any moment in the game, and observe win probability updating in real time as each snap is processed. Three architectural decisions define the application's behavior.

The first is the perspective-flip logic that enables a continuous, unbroken win probability curve across the full game. The underlying models predict the probability that the team currently in possession wins — a possession-perspective output that is optimal for learning football mechanics but creates a display problem. On a turnover, raw model output would jump discontinuously (for example, from 80% to 20%) as possession changes. The application resolves this by anchoring all displayed probabilities to the home team: when the home team has possession, the raw output is shown directly; when the away team has possession, the complement (1 minus the raw output) is displayed. The result is a single continuous line where turnovers appear as directional movements rather than vertical discontinuities, matching the broadcast format that viewers expect.

The second architectural feature is the automatic Play of the Game identification. After computing win probability for every play in a game, the application calculates the delta between consecutive plays from the winning team's perspective and flags the play with the largest positive swing as the Play of the Game. This surfaces the single most pivotal moment of the contest automatically, without requiring the user to inspect the play-by-play log manually.

The third consideration is performance. The application uses Streamlit's cache_resource decorator to load both production models and the fitted StandardScaler into memory at startup and keep them resident for the session. This eliminates per-request disk I/O that would otherwise introduce unacceptable latency. With both models cached, the application processes all plays in a typical 200-play college football game and renders the complete win probability chart in under 1.5 seconds.

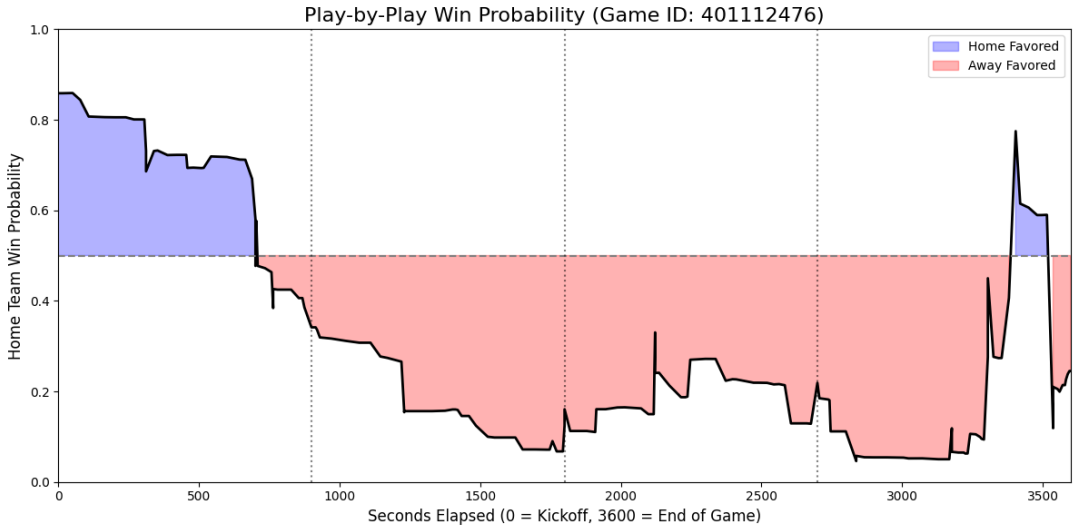

Figure 7: *Sample win probability chart generated by the application for a single game. The home team's win probability is plotted continuously across all plays, with turnovers appearing as smooth directional movements rather than abrupt jumps. The chart rises and falls naturally with scoring plays, field position changes, and time pressure, producing a broadcast-quality visualization of game momentum.*

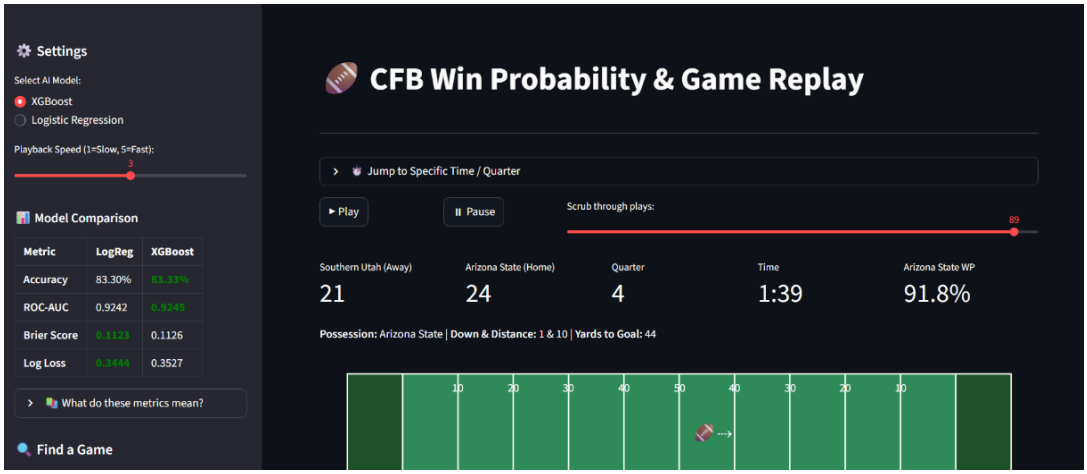

Figure 8: *The CFB Win Probability & Game Replay Streamlit application interface. The sidebar allows users to select between the XGBoost and Logistic Regression production models and compare their metrics side-by-side. The main panel shows the live game state — score, quarter, time remaining, possession, down and distance, and field position — alongside the current home team win probability (91.8% for Arizona State with 1:39 remaining in the fourth quarter in the example shown). Users can play through the game automatically or scrub to any specific play.*

Conclusion

This project demonstrates that a carefully engineered feature set, combined with principled data pipeline design, can enable a regularized linear model to achieve performance competitive with a state-of-the-art gradient boosted tree model on a 1.6-million-play dataset. The 24-feature architecture — combining raw game state, possession-perspective priors, drive momentum, time-decayed interaction terms, and polynomial threshold features — produced production models

achieving a ROC-AUC of 0.9245 to 0.9263 and accuracy of 83.34% to 83.51% across both Logistic Regression and XGBoost.

The statistical validation conducted through the statsmodels analysis confirmed that the core engineered features are genuine, highly significant predictors rather than artifacts of overfitting. inter_score_x_time returned a z-score of 129.3, making it the most statistically reliable feature in the model. The non-significant and undefined p-values for ELO and timeout features are not failures but predictable mathematical consequences of the feature construction strategy, each resolved by the L2 regularization in the production models. The redundancy paradox — where engineered features render raw features statistically redundant — is perhaps the clearest evidence that the feature engineering succeeded: the constructed signals are so dominant that the model no longer needs the raw inputs independently.

The central technical finding of this project is the power of domain-informed feature engineering. By constructing interaction terms that explicitly encode the increasing urgency of a scoring lead as the clock expires and polynomial terms that capture the non-linear safety of multi-possession advantages, the pipeline gave both models access to compound signals that neither could reliably discover from raw data alone. The consequence is that a simple regularized linear model, given the right inputs, produces calibration and discrimination metrics nearly indistinguishable from a complex ensemble of 500 gradient boosted trees. The gap between model architectures was closed not by model complexity but by the quality of the information provided to both models.

Future work on this system could extend in several directions. Adding player-level features such as quarterback completion percentage under pressure, offensive line grade, or defensive EPA by unit could capture personnel-driven momentum that the current feature set does not represent. A recurrent neural network architecture such as an LSTM, trained on the sequence of play embeddings rather than a fixed feature window, could capture longer-range momentum patterns and game flow dynamics that the five-play rolling window approximates but does not fully model. Nonetheless, the current system represents a complete, production-ready win probability engine that delivers broadcast-quality probability estimates across the full spectrum of college football games.